

Day 18 - 16th july 2025

Pending:

Heap Sort

Radix sort

Task01

What kind of collision resolution strategy is implemented in the below Hash Table ?

import java.util.*;

```
import java.util.*;

class Task01 {

    LinkedList<Entry>[] data = new LinkedList[10]; // Creating a hash table with 10 buckets 11 us

    // Method to insert key-value pairs into the hash table
    public void put(String keyval, int value) { 4 usages
        int index = Math.abs(keyval.hashCode() % data.length); // Compute hash index

        if (data[index] == null) {
            data[index] = new LinkedList<>(); // If no linked list exists, create one
        }

        // Check if the key already exists and update the value if so
        for (Entry e : data[index]) {
            if (e.keyval.equals(keyval)) {
                e.value = value; // Update the value if key is found
                return;
            }
        }

        // Add the new entry to the list
        data[index].add(new Entry(keyval, value));
    }

    // Method to retrieve the value for a given key
}
```

Activate Windows

```

public int get(String keyval) { 3 usages

    if (data[index] != null) {
        for (Entry e : data[index]) {
            if (e.keyval.equals(keyval)) {
                return e.value; // Return the value if the key is found
            }
        }
    }
    return -1; // Return -1 if the key doesn't exist
}

// Method to display the contents of the hash table
public void display() { 1 usage
    for (int i = 0; i < data.length; i++) {
        if (data[i] != null) {
            System.out.print("Bucket " + i + ": ");
            for (Entry e : data[i]) {
                System.out.print "[" + e.keyval + "=" + e.value + " ";
            }
            System.out.println();
        }
    }
}
}

```

```

        public void display() { //usage
            System.out.print("Bucket " + i + ": ");
            for (Entry e : data[i]) {
                System.out.print "[" + e.keyval + "=" + e.value + " ] ";
            }
            System.out.println();
        }
    }

    // Entry class representing a key-value pair
    static class Entry { // 5 usages
        String keyval; // 4 usages
        int value; // 4 usages

        Entry(String k, int v) { // 1 usage
            keyval = k;
            value = v;
        }
    }

    public static void main(String[] args) {
        Task01 hashTable = new Task01();

        // Adding some key-value pairs
    }
}

```

Task 02:

Wap to take input from the user a 5 digit no and display digit by digit in the output

Hint:

If input is 456897

Output:

units digit is 7

Ones digit is 9

Hundreds digit is 8

Thousands digit is 6

10 thousands digit is 5

Lakhs digit is 4

10.14 to 10.18

```
Rename usages
public class DigitByDigit1 {
    public static void main(String[] args) {
        // Create a scanner object to take input from the user
        Scanner scanner = new Scanner(System.in);

        // Ask the user for a number
        System.out.print("Enter a number: ");
        int number = scanner.nextInt();

        // Step 1: Display the digits one by one
        // Extract and display each digit from right to left
        int unitsDigit = number % 10;           // Unit place
        int tensDigit = (number / 10) % 10;      // Tens place
        int hundredsDigit = (number / 100) % 10; // Hundreds place
        int thousandsDigit = (number / 1000) % 10; // Thousands place
        int tenThousandsDigit = (number / 10000) % 10; // Ten-thousands place

        // Display the digits (Works for a number with at most 5 digits)
        System.out.println("Units digit is: " + unitsDigit);
        System.out.println("Tens digit is: " + tensDigit);
        System.out.println("Hundreds digit is: " + hundredsDigit);
        System.out.println("Thousands digit is: " + thousandsDigit);
        System.out.println("Ten-thousands digit is: " + tenThousandsDigit);
    }
}
```

Task 03:

Wap to take number from the user and display the no of digit it has

HInt:

If input is:

10,000

Output:

Its a 5 digit number

```
import java.util.Scanner;
```

 Rename usages

```
public class DigitByDigit1 {  
    public static void main(String[] args) {  
        // Create a scanner object to take input from the user  
        Scanner scanner = new Scanner(System.in);  
  
        // Ask the user for a number  
        System.out.print("Enter a number: ");  
        int number = scanner.nextInt();  
  
        // Step 1: Display the digits one by one  
        // Extract and display each digit from right to left  
        int unitsDigit = number % 10;           // Unit place  
        int tensDigit = (number / 10) % 10;      // Tens place  
        int hundredsDigit = (number / 100) % 10; // Hundreds place  
        int thousandsDigit = (number / 1000) % 10; // Thousands place  
        int tenThousandsDigit = (number / 10000) % 10; // Ten-thousands place  
  
        // Display the digits (Works for a number with at most 5 digits)  
        System.out.println("Units digit is: " + unitsDigit);  
        System.out.println("Tens digit is: " + tensDigit);  
        System.out.println("Hundreds digit is: " + hundredsDigit);  
        System.out.println("Thousands digit is: " + thousandsDigit);  
        System.out.println("Ten-thousands digit is: " + tenThousandsDigit);  
    }  
}
```

Activate Windows

35:1 Click to start your activation

```

// Step 1: Extract the digits from the number
int unitsDigit = (number / 1) % 10; // Units place
int thousandsDigit = (number / 1000) % 10; // Thousands place
int tenThousandsDigit = (number / 10000) % 10; // Ten-thousands place

// Display the digits (Works for a number with at most 5 digits)
System.out.println("Units digit is: " + unitsDigit);
System.out.println("Tens digit is: " + tensDigit);
System.out.println("Hundreds digit is: " + hundredsDigit);
System.out.println("Thousands digit is: " + thousandsDigit);
System.out.println("Ten-thousands digit is: " + tenThousandsDigit);

// Step 2: Display the number of digits
// Calculate the number of digits in the number
int digitCount = (int) Math.log10(number) + 1;

// Display the number of digits
System.out.println("This is a " + digitCount + " digit number.");
}
}

```

Activate Windows

35:1 C:\Program Files\Java\jdk-17\bin\java.exe -Djava.class.path=.

```

C:\Program Files\Java\jdk-17\bin\java.exe -Djava.class.path=
Enter a number: 12345
Units digit is: 5
Tens digit is: 4
Hundreds digit is: 3
Thousands digit is: 2
Ten-thousands digit is: 1

```

```

Thousands digit is: 2
Ten-thousands digit is: 1
This is a 5 digit number.

```

```

Process finished with exit code 0

```

```

class Task01 {
    public static void main(String[] args) {
        Task01 hashTable = new Task01();

        // Adding some key-value pairs
        hashTable.put(keyval: "Apple", value: 10);
        hashTable.put(keyval: "Banana", value: 20);
        hashTable.put(keyval: "Cherry", value: 30);
        hashTable.put(keyval: "Date", value: 40);

        // Displaying the hash table contents
        System.out.println("Hash Table Contents:");
        hashTable.display();

        // Retrieve value for a specific key
        System.out.println("\nValue for 'Banana': " + hashTable.get("Banana"));
        System.out.println("Value for 'Apple': " + hashTable.get("Apple"));
        System.out.println("Value for 'Orange': " + hashTable.get("Orange"));
    }
}

```

```

Value for 'Banana': 20
Value for 'Apple': 10
Value for 'Orange': -1

Process finished with exit code 0

```

Task 02:

Wap to take input from the user a 5 digit no and display digit by digit in the output

Hint:

If input is 456897

Output:

units digit is 7

Ones digit is 9

Hundreds digit is 8

Thousands digit is 6

10 thousands digit is 5

Lakhs digit is 4

```
import java.util.Scanner;

public class DigitByDigit {
    public static void main(String[] args) {
        // Create a scanner object to take input from the user
        Scanner scanner = new Scanner(System.in);

        // Ask the user for a 5-digit number
        System.out.print("Enter a 5-digit number: ");
        int number = scanner.nextInt();

        // Validate if the input is a 5-digit number
        if (number < 10000 || number > 99999) {
            System.out.println("Please enter a valid 5-digit number.");
            return;
        }

        // Extract and display each digit from right to left
        int unitsDigit = number % 10;           // Unit place
        int tensDigit = (number / 10) % 10;      // Tens place
        int hundredsDigit = (number / 100) % 10; // Hundreds place
        int thousandsDigit = (number / 1000) % 10; // Thousands place
        int tenThousandsDigit = (number / 10000) % 10; // Ten-thousands place

        // Display the digits
        System.out.println("Units digit is: " + unitsDigit);
        System.out.println("Tens digit is: " + tensDigit);
    }
}
```

Activate Windows


```

        // Extract and display each digit from right to left
        int unitsDigit = number % 10;           // Unit place
        int tensDigit = (number / 10) % 10;      // Tens place
        int hundredsDigit = (number / 100) % 10; // Hundreds place
        int thousandsDigit = (number / 1000) % 10; // Thousands place
        int tenThousandsDigit = (number / 10000) % 10; // Ten-thousands place

        // Display the digits
        System.out.println("Units digit is: " + unitsDigit);
        System.out.println("Tens digit is: " + tensDigit);
        System.out.println("Hundreds digit is: " + hundredsDigit);
        System.out.println("Thousands digit is: " + thousandsDigit);
        System.out.println("Ten-thousands digit is: " + tenThousandsDigit);
    }
}

```

```

Enter a 5-digit number: 12345
Units digit is: 5
Tens digit is: 4
Hundreds digit is: 3
Thousands digit is: 2
Ten-thousands digit is: 1

```

git > src >  DigitByDigit

Task 04:

Wap to display the groups of digits depending upon the unit digits

Hint:

If input is 45,81, 85,100,20. 95,60,10,21

Output:

Array 1 has : 100,20,60,10

Array 2 has : 81, 21

10.41 to 10.45

45:1 CbFto 0.8Ft+8gs 4 spaces

```

public static void main(String[] args) {
    int number = Integer.parseInt(num.trim()); // Parse the number

    // Get the unit digit using modulo 10
    int unitDigit = number % 10;

    // Add the number to the correct group based on its unit digit
    groups.get(unitDigit).add(number);
}

// Display the groups with non-empty lists
for (int i = 0; i < 10; i++) {
    if (!groups.get(i).isEmpty()) {
        System.out.println("Array " + (i + 1) + " has: " + groups.get(i));
    }
}

scanner.close(); // Close the scanner
}
}

```

Task 5

Algorithm for radix sort

Step 1:

Find the number with the most digits in the list.

This tells us how many times we need to sort (once for each digit position).

Step 2:

Start with the digit in the units place (rightmost digit).

Step 3:

Group the numbers based on the digit at that position.

For example, group all numbers that have 0 in the unit place together, then all that have 1, then 2, and so on up to 9.

Step 4:

Rearrange the numbers according to those groups.

Now the list is partially sorted based on the current digit.

Step 5:

Move to the next digit to the left (tens place), and repeat steps 3 and 4.

Step 6:

Keep repeating this process for the **hundreds place, thousands place**, etc., until you've sorted by all digit positions

Step 7:

After the final pass, the list will be **fully sorted**.

Task 6

Pseudocode for Radix sort

RadixSort(array):

 maxNumber = find the largest number in array

 digitPlace = 1

 while maxNumber / digitPlace > 0:

 sort array based on digit at digitPlace

 digitPlace = digitPlace * 10

Task 7

Full code

Rename usages

```
public class SimpleRadixSort1 {
```



```
    // Function to perform counting sort based on the current digit
```

```
    public static void countingSort(int[] arr, int exp) { 1 usage
```

```
        int n = arr.length;
```

```
        int[] output = new int[n]; // Output array
```

```
        int[] count = new int[10]; // Count array to store frequency of digit
```

```
        // Count occurrences of each digit
```

```
        for (int i = 0; i < n; i++) {
```

```
            count[(arr[i] / exp) % 10]++;
```

```
        }
```

```
        // Change count[i] to store the actual position of this digit in output
```

```
        for (int i = 1; i < 10; i++) {
```

```
            count[i] += count[i - 1];
```

```
        }
```

```
        // Build the output array using the positions in count[]
```

```
        for (int i = n - 1; i >= 0; i--) {
```

```
            output[count[(arr[i] / exp) % 10] - 1] = arr[i];
```

```
            count[(arr[i] / exp) % 10]--;
```

```
        }
```

```

public static void countingSort(int[] arr, int exp) { 1 usage
    // Build the output array using the positions in count[]
    for (int i = n - 1; i >= 0; i--) {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }

    // Copy the output array to arr[], so that arr[] contains sorted numbers
    System.arraycopy(output, 0, arr, 0, n);
}

// Main function to implement Radix Sort
public static void radixSort(int[] arr) { 1 usage
    // Find the maximum number to determine the number of digits
    int max = Arrays.stream(arr).max().getAsInt();

    // Perform counting sort for every digit (units, tens, hundreds, etc.)
    for (int exp = 1; max / exp > 0; exp *= 10) {
        countingSort(arr, exp); // Sort based on each digit
    }
}

```

```

public static void radixSort(int[] arr) { 1 usage
    // Find the maximum number to determine the number of digits
    int max = Arrays.stream(arr).max().getAsInt();

    // Perform counting sort for every digit (units, tens, hundreds, etc.)
    for (int exp = 1; max / exp > 0; exp *= 10) {
        countingSort(arr, exp); // Sort based on each digit
    }
}

public static void main(String[] args) {
    int[] arr = {170, 45, 75, 90, 802, 24, 2, 66}; // Example array

    System.out.println("Original Array: " + Arrays.toString(arr));

    // Apply Radix Sort
    radixSort(arr);

    System.out.println("Sorted Array: " + Arrays.toString(arr));
}
}

```

```
"C:\Program Files\Java\jdk-17\bin\java.exe" "-javaagent
Original Array: [170, 45, 75, 90, 802, 24, 2, 66]
Sorted Array: [2, 24, 45, 66, 75, 90, 170, 802]

Process finished with exit code 0
```

Task 08:

Do you find any significance change between the `breadthFirstSearchRecursive()` approach compared to the standard BFS?

Yes, there is a significant change between the two approaches.

Standard Breadth-First Search (BFS) is defined as a graph traversal technique that uses a queue to explore nodes level by level, starting from a given source node. It is naturally implemented using an iterative approach.

In contrast, `breadthFirstSearchRecursive()` is an unconventional approach that attempts to perform BFS using recursion, which relies on the call stack instead of explicit looping.

Standard BFS uses a queue data structure for level-order traversal.

Recursive BFS uses recursive function calls to simulate the same process.

Standard BFS is more efficient, widely accepted, and aligns with the definition of BFS.

Recursive BFS deviates from this standard definition and is more complex, less practical, and can cause performance issues due to deep recursion.

Will it achieve the same result but emphasizes recursive style using the same level-order logic with explicit queue management?

Yes!

- Recursive BFS emulates the same level-order traversal.
- It still uses an explicit queue, just managed through recursive function calls.
- The main difference is the style of implementation (recursive vs iterative), but the logic remains the same.

This is the correct statement.

Task 09:

What is memoization?

Write in own words with examples.

Memoization is like keeping a cheat sheet for your program. When your program solves a small problem, it writes down the answer. Later, if it needs to solve the same small problem again, it just looks at the cheat sheet instead of solving it all over. This saves a lot of time because it doesn't repeat the same work.

For example, if you want to find the 5th number in the Fibonacci sequence, you normally have to find the 4th and 3rd numbers first. But finding the 3rd number might happen many times. With memoization, once you find the 3rd number, you save it. Next time you need it, you just use the saved answer.

So, memoization helps your program remember past answers to avoid doing extra work and run faster.

Example:

Imagine you want to find the Fibonacci number for 5.

- Normally, you calculate Fibonacci(5) by calculating Fibonacci(4) and Fibonacci(3).
- Then to calculate Fibonacci(4), you calculate Fibonacci(3) and Fibonacci(2), and so on.
- But you see Fibonacci(3) gets calculated multiple times.

With memoization, once you calculate Fibonacci(3) once, you save it in memory. Next time you need Fibonacci(3), you just return the saved value instead of calculating again.

Task 10:

What do you understand by Dynamic Programming

Write in your own words with examples.

Dynamic programming is like solving a big problem by breaking it into smaller parts, solving each part only once, and then remembering those solutions to use them again whenever needed. Instead of repeating the same work over and over, you store answers to smaller problems and build up to the final answer efficiently.

It's especially useful when a problem has overlapping subproblems — meaning many parts of the problem repeat themselves. By saving the answers to these repeated parts, you avoid wasting time.

Think of it like solving a puzzle step-by-step, remembering which pieces fit where, so you don't try the same piece in the same place multiple times.

Imagine you want to climb a staircase with 5 steps, and you can take either 1 step or 2 steps at a time. To find out how many ways there are to reach the top, you could think:

- To get to step 5, you could come from step 4 or step 3.
- To get to step 4, you could come from step 3 or step 2.
- And so on...

Instead of calculating the ways to get to step 3 and 4 multiple times, dynamic programming helps you remember those answers so when you need them again, you just use the saved result. This makes the problem easier and faster to solve.

Task 11:

Can you write fibonacci series using dynamic programming?

```
public class FibonacciDP { no usages

    // Function to calculate Fibonacci series up to n using dynamic programming (bottom-up approach)
    public static int fibonacci(int n) {
        // If n is 0 or 1, return n as the result (base cases)
        if (n == 0) return 0;
        if (n == 1) return 1;

        // Create an array to store Fibonacci numbers up to n
        int[] dp = new int[n + 1];

        // Base cases
        dp[0] = 0;
        dp[1] = 1;

        // Calculate Fibonacci numbers from 2 to n
        for (int i = 2; i <= n; i++) {
            dp[i] = dp[i - 1] + dp[i - 2]; // Fibonacci relation
        }

        return dp[n]; // Return the nth Fibonacci number
    }

    public static void main(String[] args) {
        int n = 10; // Example value for n
        System.out.println("Fibonacci number at index " + n + " is: " + fibonacci(n));
    }
}
```

Activate Windows

```

public static int fibonacci(int n) {

    // Base cases
    dp[0] = 0;
    dp[1] = 1;

    // Calculate Fibonacci numbers from 2 to n
    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2]; // Fibonacci relation
    }

    return dp[n]; // Return the nth Fibonacci number
}

public static void main(String[] args) {
    int n = 10; // Example value for n
    System.out.println("Fibonacci number at index " + n + " is: " + fibonacci(n));
}
}

```

```

C:\Program Files\Java\jdk-17\bin\java.exe
Fibonacci number at index 10 is: 55

Process finished with exit code 0

```

TSK 13:

how can you say recursive functions maintain the state of each call during execution?

1. Each recursive call creates a new thread, and context switching maintains state.
2. Recursive functions store state in global variables accessible across calls.
3. The system call stack tracks local variables and return addresses for each recursive invocation.
4. Recursive functions replicate the heap structure to keep values between calls.

Task 14:

Iterative implementations use less memory as they do not require stack frames for each call.
True

with regard to the difference in memory usage between recursive and iterative implementations of the same algorithm?
the above statement is true or false

True

Task 15:

Recursion that lacks a proper base case or makes too many nested calls, exhausting the call stack. – true

with regard to stack overflow error in recursive functions.. do you think above statement is true

Task 16:

what happens when inserting keys with the same hash in this custom hash map

```
public class HashCollision {  
  
    static class Entry {  
        String key;  
        int value;  
  
        Entry(String key, int value) {  
            this.key = key;  
            this.value = value;  
        }  
    }  
  
    List<Entry>[] table = new AL[10];  
  
    public void put(String key, int val) {  
        int index = Math.abs(key.hashCode() % table.length);
```

```

        if (table[index] == null) {
            table[index] = new AL<>();
        }

        table[index].add(new Entry(key, val));
    }
}

```

1. Insertion will fail due to duplicate key exception
2. Values are distributed across different buckets using linear probing
3. Only one key-value pair will be stored due to overwriting
4. Multiple values are stored in same bucket via chaining

Task 17:

Write a code for binary search tree for expected output

```
import java.util.*;

class Node { 12 usages
    int key; 2 usages
    Node left, right; 8 usages

    public Node(int key) { 7 usages
        this.key = key;
        left = right = null;
    }
}

class BinaryTreeCornerNodes {
    Node root; 8 usages

    void printCorner(Node root) { 1 usage
        if (root == null) return;

        Queue<Node> q = new LinkedList<>();
        q.add(root);

        while (!q.isEmpty()) {
            int n = q.size();

            for (int i = 0; i < n; i++) {
                Node temp = q.poll();
            }
        }
    }
}
```

```

        // Print the first and last node of the current level
        if (i == 0 || i == n - 1) {
            System.out.print(temp.key + " ");
        }

        if (temp.left != null)
            q.add(temp.left);
        if (temp.right != null)
            q.add(temp.right);
    }
}

```

```

public static void main(String[] args) {
    BinaryTreeCornerNodes tree = new BinaryTreeCornerNodes();

    // Build the tree
    tree.root = new Node( key: 11);
    tree.root.left = new Node( key: 22);
    tree.root.right = new Node( key: 33);
    tree.root.left.left = new Node( key: 44);
    tree.root.left.right = new Node( key: 55);
    tree.root.right.left = new Node( key: 66);
    tree.root.right.right = new Node( key: 77);
}

```

```
public static void main(String[] args) {  
    BinaryTreeCornerNodes tree = new BinaryTreeCornerNodes();  
  
    // Build the tree  
    tree.root = new Node(key: 11);  
    tree.root.left = new Node(key: 22);  
    tree.root.right = new Node(key: 33);  
    tree.root.left.left = new Node(key: 44);  
    tree.root.left.right = new Node(key: 55);  
    tree.root.right.left = new Node(key: 66);  
    tree.root.right.right = new Node(key: 77);  
  
    // Print corner nodes at each level  
    tree.printCorner(tree.root);  
}
```

```
"C:\Program Files\Java\jdk-17\bin\java.exe"  
11 22 33 44 77  
Process finished with exit code 0
```